

Library Management System

Java Console Application

Java, core language only (no external libraries). Console app with a numeric menu. Data is saved to text files so it persists between runs.

Files: Book.java, Member.java, Transaction.java, LibraryException.java, FileManager.java, Library.java, LibraryManagementSystem.java

What it does

Manages a book catalog and a list of members, and lets a librarian issue and return books. Each loan gets a 14-day due date, and returning a book late calculates a fine (5 per day late). Everything is saved to plain text files on exit and loaded back in on the next run.

Menu options

- Add a book (or add more copies of one that already exists)
- Remove a book (blocked if copies are still on loan)
- List all books / search by title
- Register a member
- List members
- Issue a book to a member
- Return a book (shows fine if late)
- View a member's active loans
- View all overdue books

Classes

Book.java	Book data: ISBN, title, author, copy counts
Member.java	Member data: ID, name, email
Transaction.java	One issue/return record with dates
LibraryException.java	Custom exception for error cases (book not found, no copies left, etc.)
FileManager.java	Reads/writes the text files
Library.java	All the actual logic - issuing, returning, fines, checks
LibraryManagementSystem.java	Main class - the menu and user input

How to run

Put all seven .java files in one folder, then:

```
javac *.java  
java LibraryManagementSystem
```

Pick a number from the menu. Option 0 saves everything to books.txt, members.txt and transactions.txt in the same folder.

Source Code

Book.java

```
import java.io.Serializable;

/**
 * Represents a single book in the library catalog.
 */
public class Book implements Serializable {

    private static final long serialVersionUID = 1L;

    private String isbn;
    private String title;
    private String author;
    private int totalCopies;
    private int availableCopies;

    public Book(String isbn, String title, String author, int totalCopies) {
        this.isbn = isbn;
        this.title = title;
        this.author = author;
        this.totalCopies = totalCopies;
        this.availableCopies = totalCopies;
    }

    // ----- Getters / Setters -----
    public String getIsbn() {
        return isbn;
    }

    public String getTitle() {
        return title;
    }

    public String getAuthor() {
        return author;
    }

    public int getTotalCopies() {
```

```

        return totalCopies;
    }

    public int getAvailableCopies() {
        return availableCopies;
    }

    public void setTotalCopies(int totalCopies) {
        this.totalCopies = totalCopies;
    }

    public void incrementAvailable() {
        if (this.availableCopies < this.totalCopies) {
            this.availableCopies++;
        }
    }

    public void decrementAvailable() {
        if (this.availableCopies > 0) {
            this.availableCopies--;
        }
    }

    public boolean isAvailable() {
        return availableCopies > 0;
    }

    @Override
    public String toString() {
        return String.format("%-12s | %-30s | %-20s | Total:%-3d | Available:%-3d",
            isbn, title, author, totalCopies, availableCopies);
    }

    /**
     * Serializes this book to a pipe-delimited line for file storage.
     */
    public String toFileString() {
        return isbn + "|" + title + "|" + author + "|" + totalCopies + "|" + availableCopies;
    }

    /**

```

```

    * Rebuilds a Book object from a pipe-delimited file line.
    */
    public static Book fromFileString(String line) {
        String[] parts = line.split("\\|");
        Book book = new Book(parts[0], parts[1], parts[2], Integer.parseInt(parts[3]));
        book.availableCopies = Integer.parseInt(parts[4]);
        return book;
    }
}

```

Member.java

```

import java.io.Serializable;

/**
 * Represents a library member who can borrow books.
 */
public class Member implements Serializable {

    private static final long serialVersionUID = 1L;

    private String memberId;
    private String name;
    private String email;

    public Member(String memberId, String name, String email) {
        this.memberId = memberId;
        this.name = name;
        this.email = email;
    }

    public String getMemberId() {
        return memberId;
    }

    public String getName() {
        return name;
    }

    public String getEmail() {

```

```

        return email;
    }

    @Override
    public String toString() {
        return String.format("%-10s | %-20s | %-25s", memberId, name, email);
    }

    public String toFileString() {
        return memberId + "|" + name + "|" + email;
    }

    public static Member fromFileString(String line) {
        String[] parts = line.split("\\|");
        return new Member(parts[0], parts[1], parts[2]);
    }
}

```

Transaction.java

```

import java.io.Serializable;
import java.time.LocalDate;

/**
 * Represents a single book-issue transaction (issued or already returned).
 */
public class Transaction implements Serializable {

    private static final long serialVersionUID = 1L;

    private String isbn;
    private String memberId;
    private LocalDate issueDate;
    private LocalDate dueDate;
    private LocalDate returnDate; // null while the book is still out

    public Transaction(String isbn, String memberId, LocalDate issueDate, LocalDate dueDate) {
        this.isbn = isbn;
        this.memberId = memberId;
        this.issueDate = issueDate;
    }
}

```

```

        this.dueDate = dueDate;
        this.returnDate = null;
    }

    public String getIsbn() {
        return isbn;
    }

    public String getMemberId() {
        return memberId;
    }

    public LocalDate getIssueDate() {
        return issueDate;
    }

    public LocalDate getDueDate() {
        return dueDate;
    }

    public LocalDate getReturnDate() {
        return returnDate;
    }

    public boolean isReturned() {
        return returnDate != null;
    }

    public void markReturned(LocalDate returnDate) {
        this.returnDate = returnDate;
    }

    public String toFileString() {
        return isbn + "|" + memberId + "|" + issueDate + "|" + dueDate + "|"
            + (returnDate == null ? "NULL" : returnDate);
    }

    public static Transaction fromFileString(String line) {
        String[] p = line.split("\\|");
        Transaction t = new Transaction(p[0], p[1], LocalDate.parse(p[2]), LocalDate.parse(p[3]));
        if (!p[4].equals("NULL")) {

```



```

        t.markReturned(LocalDate.parse(p[4]));
    }
    return t;
}

@Override
public String toString() {
    String status = isReturned() ? "Returned on " + returnDate : "Due on " + dueDate;
    return String.format("ISBN:%-10s Member:%-8s Issued:%-12s %s",
        isbn, memberId, issueDate, status);
}
}

```

LibraryException.java

```

/**
 * Custom checked exception used to signal domain-specific
 * error conditions inside the Library Management System
 * (e.g. book not found, no copies available, member not found).
 */
public class LibraryException extends Exception {

    public LibraryException(String message) {
        super(message);
    }
}

```

FileManager.java

```

import java.io.*;
import java.util.*;

/**
 * Handles all file persistence (load/save) for books, members and
 * transactions. Uses simple pipe-delimited text files so the data
 * is human-readable and needs no external libraries.
 */
public class FileManager {

    private static final String BOOKS_FILE = "books.txt";

```

```

private static final String MEMBERS_FILE = "members.txt";
private static final String TRANSACTIONS_FILE = "transactions.txt";

public List<Book> loadBooks() {
    List<Book> books = new ArrayList<>();
    books.addAll(readLines(BOOKS_FILE, Book::fromFileString));
    return books;
}

public List<Member> loadMembers() {
    return readLines(MEMBERS_FILE, Member::fromFileString);
}

public List<Transaction> loadTransactions() {
    return readLines(TRANSACTIONS_FILE, Transaction::fromFileString);
}

public void saveBooks(Collection<Book> books) {
    writeLines(BOOKS_FILE, books, Book::toFileString);
}

public void saveMembers(Collection<Member> members) {
    writeLines(MEMBERS_FILE, members, Member::toFileString);
}

public void saveTransactions(Collection<Transaction> transactions) {
    writeLines(TRANSACTIONS_FILE, transactions, Transaction::toFileString);
}

// ----- Generic helpers -----

private <T> List<T> readLines(String fileName,
                             java.util.function.Function<String, T> parser) {
    List<T> result = new ArrayList<>();
    File file = new File(fileName);
    if (!file.exists()) {
        return result; // nothing saved yet - first run
    }
    try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
        String line;
        while ((line = reader.readLine()) != null) {

```

```

        if (!line.trim().isEmpty()) {
            result.add(parser.apply(line));
        }
    }
} catch (IOException e) {
    System.out.println("Warning: could not read " + fileName
        + " (" + e.getMessage() + ")");
}
return result;
}

private <T> void writeLines(String fileName, Collection<T> items,
    java.util.function.Function<T, String> toLine) {
    try (BufferedWriter writer = new BufferedWriter(new FileWriter(fileName))) {
        for (T item : items) {
            writer.write(toLine.apply(item));
            writer.newLine();
        }
    } catch (IOException e) {
        System.out.println("Warning: could not save " + fileName
            + " (" + e.getMessage() + ")");
    }
}
}
}

```

Library.java

```

import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.*;

/**
 * Core business logic of the Library Management System.
 * Keeps books, members and transactions in memory (backed by
 * Collections) and persists them to disk through FileManager.
 */
public class Library {

    private static final int LOAN_PERIOD_DAYS = 14;
    private static final double FINE_PER_DAY = 5.0; // currency units per overdue day

```

```

private final Map<String, Book> books = new LinkedHashMap<>(); // isbn -> Book
private final Map<String, Member> members = new LinkedHashMap<>(); // memberId -> Member
private final List<Transaction> transactions = new ArrayList<>();

private final FileManager fileManager = new FileManager();

public Library() {
    loadData();
}

// -----
// Book operations
// -----

public void addBook(String isbn, String title, String author, int copies)
    throws LibraryException {
    if (books.containsKey(isbn)) {
        // Book already exists -> just add more copies
        Book existing = books.get(isbn);
        existing.setTotalCopies(existing.getTotalCopies() + copies);
        for (int i = 0; i < copies; i++) {
            existing.incrementAvailable();
        }
    } else {
        if (isbn == null || isbn.isBlank()) {
            throw new LibraryException("ISBN cannot be empty.");
        }
        books.put(isbn, new Book(isbn, title, author, copies));
    }
}

public void removeBook(String isbn) throws LibraryException {
    Book book = getBookOrThrow(isbn);
    if (book.getAvailableCopies() != book.getTotalCopies()) {
        throw new LibraryException("Cannot remove '" + book.getTitle()
            + "' - some copies are still issued.");
    }
    books.remove(isbn);
}

public List<Book> searchByTitle(String keyword) {

```

```

        List<Book> result = new ArrayList<>();
        for (Book b : books.values()) {
            if (b.getTitle().toLowerCase().contains(keyword.toLowerCase())) {
                result.add(b);
            }
        }
        return result;
    }

    public List<Book> getAllBooks() {
        return new ArrayList<>(books.values());
    }

    // -----
    // Member operations
    // -----

    public void registerMember(String memberId, String name, String email)
        throws LibraryException {
        if (members.containsKey(memberId)) {
            throw new LibraryException("A member with ID '" + memberId + "' already exists.");
        }
        members.put(memberId, new Member(memberId, name, email));
    }

    public List<Member> getAllMembers() {
        return new ArrayList<>(members.values());
    }

    // -----
    // Issue / Return operations
    // -----

    public Transaction issueBook(String isbn, String memberId) throws LibraryException {
        Book book = getBookOrThrow(isbn);
        getMemberOrThrow(memberId);

        if (!book.isAvailable()) {
            throw new LibraryException("No copies of '" + book.getTitle()
                + "' are currently available.");
        }
    }

```

```

        boolean alreadyHasCopy = transactions.stream()
            .anyMatch(t -> !t.isReturned()
                && t.getIsbn().equals(isbn)
                && t.getMemberId().equals(memberId));
        if (alreadyHasCopy) {
            throw new LibraryException("This member already has a copy of that book checked out.");
        }

        LocalDate issueDate = LocalDate.now();
        LocalDate dueDate = issueDate.plusDays(LOAN_PERIOD_DAYS);
        Transaction transaction = new Transaction(isbn, memberId, issueDate, dueDate);

        book.decrementAvailable();
        transactions.add(transaction);
        return transaction;
    }

    public double returnBook(String isbn, String memberId) throws LibraryException {
        Book book = getBookOrThrow(isbn);
        getMemberOrThrow(memberId);

        Transaction transaction = transactions.stream()
            .filter(t -> !t.isReturned()
                && t.getIsbn().equals(isbn)
                && t.getMemberId().equals(memberId))
            .findFirst()
            .orElseThrow(() -> new LibraryException(
                "No active loan found for member '" + memberId
                    + "' on book '" + isbn + "'."));

        LocalDate returnDate = LocalDate.now();
        transaction.markReturned(returnDate);
        book.incrementAvailable();

        long overdueDays = ChronoUnit.DAYS.between(transaction.getDueDate(), returnDate);
        double fine = overdueDays > 0 ? overdueDays * FINE_PER_DAY : 0.0;
        return fine;
    }

    public List<Transaction> getActiveTransactionsForMember(String memberId) {
        List<Transaction> result = new ArrayList<>();
    }

```

```

        for (Transaction t : transactions) {
            if (!t.isReturned() && t.getMemberId().equals(memberId)) {
                result.add(t);
            }
        }
        return result;
    }

    public List<Transaction> getAllTransactions() {
        return new ArrayList<>(transactions);
    }

    public List<Transaction> getOverdueTransactions() {
        List<Transaction> result = new ArrayList<>();
        LocalDate today = LocalDate.now();
        for (Transaction t : transactions) {
            if (!t.isReturned() && t.getDueDate().isBefore(today)) {
                result.add(t);
            }
        }
        return result;
    }

    // -----
    // Persistence
    // -----

    private void loadData() {
        for (Book b : fileManager.loadBooks()) {
            books.put(b.getIsbn(), b);
        }
        for (Member m : fileManager.loadMembers()) {
            members.put(m.getMemberId(), m);
        }
        transactions.addAll(fileManager.loadTransactions());
    }

    public void saveData() {
        fileManager.saveBooks(books.values());
        fileManager.saveMembers(members.values());
        fileManager.saveTransactions(transactions);
    }

```

```

    }

    // -----
    // Helpers
    // -----

    private Book getBookOrThrow(String isbn) throws LibraryException {
        Book book = books.get(isbn);
        if (book == null) {
            throw new LibraryException("No book found with ISBN '" + isbn + "'.");
        }
        return book;
    }

    private Member getMemberOrThrow(String memberId) throws LibraryException {
        Member member = members.get(memberId);
        if (member == null) {
            throw new LibraryException("No member found with ID '" + memberId + "'.");
        }
        return member;
    }
}

```

LibraryManagementSystem.java

```

import java.util.List;
import java.util.Scanner;

/**
 * Entry point of the application. Provides a simple menu-driven
 * console interface for interacting with the Library.
 */
public class LibraryManagementSystem {

    private static final Scanner scanner = new Scanner(System.in);
    private static final Library library = new Library();

    public static void main(String[] args) {
        System.out.println("=====");
        System.out.println("        LIBRARY MANAGEMENT SYSTEM (Java)        ");
    }
}

```



```

System.out.println("=====");

boolean running = true;
while (running) {
    printMenu();
    String choice = scanner.nextLine().trim();
    try {
        switch (choice) {
            case "1": addBook(); break;
            case "2": removeBook(); break;
            case "3": listBooks(); break;
            case "4": searchBooks(); break;
            case "5": registerMember(); break;
            case "6": listMembers(); break;
            case "7": issueBook(); break;
            case "8": returnBook(); break;
            case "9": viewMemberLoans(); break;
            case "10": viewOverdueBooks(); break;
            case "0":
                library.saveData();
                System.out.println("Data saved. Goodbye!");
                running = false;
                break;
            default:
                System.out.println("Invalid choice, please try again.");
        }
    } catch (LibraryException e) {
        System.out.println("Error: " + e.getMessage());
    } catch (NumberFormatException e) {
        System.out.println("Error: please enter a valid number.");
    }
}
scanner.close();
}

private static void printMenu() {
    System.out.println("\n--- MENU ---");
    System.out.println(" 1. Add Book");
    System.out.println(" 2. Remove Book");
    System.out.println(" 3. List All Books");
    System.out.println(" 4. Search Book by Title");
}

```

```

        System.out.println(" 5. Register Member");
        System.out.println(" 6. List All Members");
        System.out.println(" 7. Issue Book");
        System.out.println(" 8. Return Book");
        System.out.println(" 9. View Member's Active Loans");
        System.out.println("10. View Overdue Books");
        System.out.println(" 0. Save & Exit");
        System.out.print("Enter choice: ");
    }

    private static void addBook() throws LibraryException {
        System.out.print("ISBN: ");
        String isbn = scanner.nextLine().trim();
        System.out.print("Title: ");
        String title = scanner.nextLine().trim();
        System.out.print("Author: ");
        String author = scanner.nextLine().trim();
        System.out.print("Number of copies: ");
        int copies = Integer.parseInt(scanner.nextLine().trim());

        library.addBook(isbn, title, author, copies);
        System.out.println("Book added/updated successfully.");
    }

    private static void removeBook() throws LibraryException {
        System.out.print("ISBN to remove: ");
        String isbn = scanner.nextLine().trim();
        library.removeBook(isbn);
        System.out.println("Book removed successfully.");
    }

    private static void listBooks() {
        List<Book> books = library.getAllBooks();
        if (books.isEmpty()) {
            System.out.println("No books in the catalog yet.");
            return;
        }
        System.out.println("\nISBN          | Title          | Author          | Copies");
        System.out.println("-".repeat(80));
        for (Book b : books) {
            System.out.println(b);
        }
    }

```

```

    }
}

private static void searchBooks() {
    System.out.print("Enter title keyword: ");
    String keyword = scanner.nextLine().trim();
    List<Book> results = library.searchByTitle(keyword);
    if (results.isEmpty()) {
        System.out.println("No matching books found.");
    } else {
        results.forEach(System.out::println);
    }
}

private static void registerMember() throws LibraryException {
    System.out.print("Member ID: ");
    String id = scanner.nextLine().trim();
    System.out.print("Name: ");
    String name = scanner.nextLine().trim();
    System.out.print("Email: ");
    String email = scanner.nextLine().trim();

    library.registerMember(id, name, email);
    System.out.println("Member registered successfully.");
}

private static void listMembers() {
    List<Member> members = library.getAllMembers();
    if (members.isEmpty()) {
        System.out.println("No members registered yet.");
        return;
    }
    System.out.println("\nMember ID    | Name                                | Email");
    System.out.println("-".repeat(60));
    for (Member m : members) {
        System.out.println(m);
    }
}

private static void issueBook() throws LibraryException {
    System.out.print("ISBN: ");

```

```

        String isbn = scanner.nextLine().trim();
        System.out.print("Member ID: ");
        String memberId = scanner.nextLine().trim();

        Transaction t = library.issueBook(isbn, memberId);
        System.out.println("Book issued. Due date: " + t.getDueDate());
    }

    private static void returnBook() throws LibraryException {
        System.out.print("ISBN: ");
        String isbn = scanner.nextLine().trim();
        System.out.print("Member ID: ");
        String memberId = scanner.nextLine().trim();

        double fine = library.returnBook(isbn, memberId);
        if (fine > 0) {
            System.out.printf("Book returned late. Fine due: %.2f%n", fine);
        } else {
            System.out.println("Book returned on time. No fine.");
        }
    }

    private static void viewMemberLoans() {
        System.out.print("Member ID: ");
        String memberId = scanner.nextLine().trim();
        List<Transaction> active = library.getActiveTransactionsForMember(memberId);
        if (active.isEmpty()) {
            System.out.println("This member has no active loans.");
        } else {
            active.forEach(System.out::println);
        }
    }

    private static void viewOverdueBooks() {
        List<Transaction> overdue = library.getOverdueTransactions();
        if (overdue.isEmpty()) {
            System.out.println("No overdue books. Great job!");
        } else {
            overdue.forEach(System.out::println);
        }
    }
}

```

}

Sample Run

```
=====
      LIBRARY MANAGEMENT SYSTEM (Java)
=====
```

--- MENU ---

1. Add Book

...

Enter choice: 1

ISBN: 978-0134685991

Title: Effective Java

Author: Joshua Bloch

Number of copies: 3

Book added/updated successfully.

Enter choice: 1

ISBN: 978-0596009205

Title: Head First Design Patterns

Author: Eric Freeman

Number of copies: 2

Book added/updated successfully.

Enter choice: 3

ISBN	Title	Author	Copies
------	-------	--------	--------

978-0134685991	Effective Java	Joshua Bloch	Total:3 Available:3
978-0596009205	Head First Design Patterns	Eric Freeman	Total:2 Available:2

Enter choice: 5

Member ID: M001

Name: Alice Johnson

Email: alice@example.com

Member registered successfully.

Enter choice: 5

Member ID: M002

Name: Bob Smith

Email: bob@example.com

Member registered successfully.

Enter choice: 7

ISBN: 978-0134685991
Member ID: M001
Book issued. Due date: 2026-08-22

Enter choice: 7
ISBN: 978-0134685991
Member ID: M001
Error: This member already has a copy of that book checked out.

Enter choice: 7
ISBN: 978-0596009205
Member ID: M002
Book issued. Due date: 2026-08-22

Enter choice: 9
Member ID: M001
ISBN:978-0134685991 Member:M001 Issued:2026-08-08 Due on 2026-08-22

Enter choice: 8
ISBN: 978-0134685991
Member ID: M001
Book returned on time. No fine.

Enter choice: 10
No overdue books. Great job!

Enter choice: 4
Enter title keyword: Design
978-0596009205 | Head First Design Patterns | Eric Freeman | Total:2 | Available:1

Enter choice: 0
Data saved. Goodbye!